OPERATING SYSTEMS – ASSIGNMENT 1

TASK 7 – Evaluate Process Control System Calls (4 Marks)
CO1–CO5 | Bloom's Level: K5 – Evaluate

Introduction

A command shell starts programs and manages their execution. Process-control system calls allow the shell to create a child process, run a command and collect its exit status.

1. Functions of fork(), exec(), wait() and exit()

fork(): Creates a child process by duplicating the calling process. The child receives a return value of 0, while the parent receives the child process ID.
exec(): Replaces the current process image with a new program. The process ID remains the same if execution succeeds.
wait() / waitpid(): Makes the parent wait for a child process to finish and collects its termination status.
exit() / _exit(): Terminates a process and provides an exit status to the parent. exit() performs C library cleanup and flushes standard I/O streams; _exit() terminates without the normal stdio cleanup.

2. How a shell runs a command

1. The shell reads and interprets the command entered by the user.
2. It calls fork() to create a child process.
3. The child calls an exec function to replace itself with the requested command.
4. The parent shell calls waitpid() to wait for the child.
5. When the child finishes, the parent obtains its exit status and displays or uses it.

3. Zombie and orphan processes

A **zombie** is a child process that has terminated but whose parent has not yet collected its exit status. The OS keeps a small process-table entry until the parent calls wait() or waitpid(). An **orphan** is a child whose parent terminates before it does. The OS reassigns the orphan to a system process that can collect its termination status.

4. Why fork() followed by exec() is used

Separating process creation and program execution gives the shell flexibility. After fork(), the child can change its input/output redirection, set up pipes or change other process attributes before calling exec(). A single create-process operation would not provide the same flexible preparation steps.

5. Completed C program

#define _POSIX_C_SOURCE 200809L #include #include #include int main(void) { printf("shell: running job 1\n"); fflush(stdout); pid_t pid = fork(); if (pid < 0) { perror("fork"); return 1; } if (pid == 0) { execlp("expr", "expr", "6", "*", "7", (char *)NULL); perror("execlp"); _exit(127); } int status; if (waitpid(pid, &status;, 0) < 0) { perror("waitpid"); return 1; } if (WIFEXITED(status)) printf("job 1 exit status %d\n", WEXITSTATUS(status)); printf("shell: running job 2\n"); fflush(stdout); pid = fork(); if (pid < 0) { perror("fork"); return 1; } if (pid == 0) _exit(3); waitpid(pid, &status;, 0); if (WIFEXITED(status)) printf("job 2 exit status %d\n", WEXITSTATUS(status)); return 0; }

6. Compile and expected output

Compile using **gcc -std=c11 -Wall -Wextra process.c -o process** and run using **./process**. Expected output:
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3

The command expr prints 42 and exits with status 0. The second child exits with status 3.

7. Why stdout is flushed before fork()

When stdout is redirected to a file, it is usually buffered. If the buffer contains text when fork() copies the process, both parent and child may hold a copy of that buffered text. If both later flush it, the same line can appear twice. Calling fflush(stdout) before fork() empties the buffer, preventing the duplicate. The assignment asks to remove the second fflush() and change _exit(3) to exit(3); exit() flushes the copied stdio buffer, so the line may be written by both processes.

8. Alternative concept: posix_spawn() or threads

posix_spawn(): It provides a combined way to create a process and start a program. It can be more efficient than fork() followed by exec() on some systems, especially when the parent process is large. It still creates a separate process, so program memory and resources are isolated.
Threads: Threads share the same process address space and many resources, so creating them can use less memory and time than creating separate processes. However, threads are not a direct replacement for independent shell commands because they share memory and can affect one another. For unrelated commands, process isolation is safer. The exact time to start 1,000 processes or threads must be measured on the test computer.

Conclusion

fork(), exec() and waitpid() allow a shell to start commands and collect their exit status. Process isolation is useful for running independent programs safely.

Prepared from the corresponding task in the uploaded CSA04 Operating Systems Assignment 1 document. Add your name, register number, required screenshots and your own measured evidence wherever requested before submission.